

데이터 분석 및 시각화 학습 자료

✓ 시각화 인사이트

0. 파이썬 기본 문법

- a. 데이터 타입별 접근
- b. 시퀀스 데이터 접근 방법 (list)
- c. 함수 선언 및 사용

1. 라이브러리 사용

- a. 넘파이
 - i. 기초 통계량 함수 사용
 - ii. 조건필터링
 - iii. 그룹별 계산

b. 판다스

2. 데이터 전처리

- a. 중복 함수 처리
 - i. duplicated
- b. 결측치 찾기 & 제거
 - i. isnull().mean() 비율
 - ii. fillna
- c. 이상치 찾기 & 제거
 - i. boxplot
 - ii. IQR 정의
 - 1. quantile(0.25)~(0.75)
- d. 스케일링
 - i. z-score
 - ii. 값-최소/최대-최소
- e. 범주형 데이터 인코딩
 - i. pd.get_dummies(칼럼)
- f. 그룹별 연산

- i. `df_group = df.groupby(['칼럼'])['칼럼'].mean()`
 - ii. `df_group = df_group.reset_index()`
- g. 파생변수 생성
- h. 데이터 타입 변경하기
 - 날짜, 범주, 연속형 변수로 바꾸는 방법
 - 0,1이면 연속형이어도 범주형으로 바뀌어야함
- i. **특정 칼럼 추가하기**
 - i. `np.where(조건, '값1', np.where(조건, '값2', '값3))`
 - ii. `apply`를 활용한 조건 함수식 적용하기
- j. 특정 칼럼 지우기
 - i. `df.drop(index=[1,3])` # 행지우기
 - 1. `df = df.drop(df[df['Age'] < 20].index)` (조건 추가)
 - ii. `df = df.drop(columns=['열1', '열2'])` # 열지우기
- k. 조건별 데이터 필터링하기
 - `iloc`
 - `bool`연산자
 - 조건연산자

Seaborn

- 시각화 라이브러리로 `matplotlib`와 더불어 활용도가 높은 시각화 라이브러리
- `plt` 보다 다양한 그래프를 표현 가능하고 인자를 설정가능함!!
 - `plt`로 생성할 수 있는 시각화 자료와 `sns`를 이용한 시각화 자료는 차이가 있음!!

시각화 실습(sns)

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_diabetes
```

Scatterplot & boxplot (산점도와 박스플롯)

| 주요 변수와 목표값과의 **관계 분석**을 할 때 사용!!

- 박스플롯과 산점도는 선형, 비선형 관계를 알 수 있으며 상관관계의 방향성을 확인할 수 있음
- subplot : boxplot, scatterplot
 - 하나의 목표 변수와 여러개의 주요 변수를 그리고자 할때 subplot
 - 기본적으로 하나만 출력할 때는 안써도 됨

참고 sns.subplots과 plt.figure()

- plt.figure : heatmap
 - 전체 그림을 그리고자 할 때

```
# 산점도와 boxplot을 하려면 sns.subplots로 도화지 만들기
# fig : 액자 사이즈
# axes 차원 3x2 사이즈 만들기

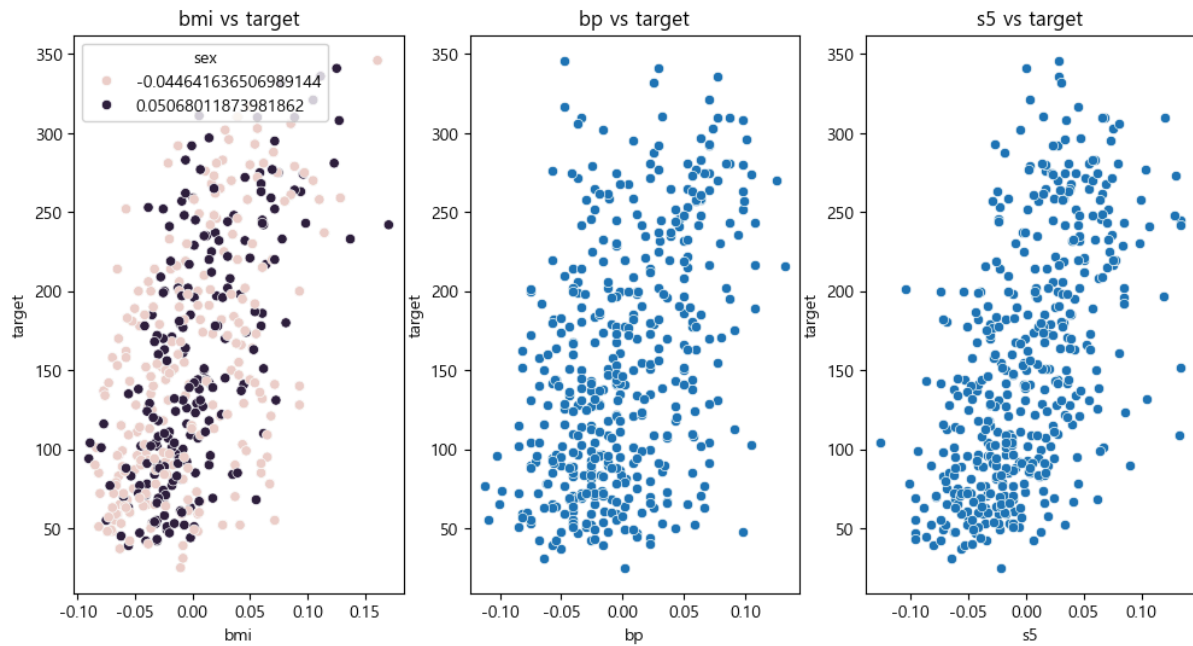
fig, axes = plt.subplots(1, 3, figsize=(12, 6))

sns.scatterplot(x='bmi', y='target', data=df, ax=axes[0], hue='sex')
axes[0].set_title('bmi vs target')

sns.scatterplot(x='bp', y='target', data=df, ax=axes[1])
axes[1].set_title('bp vs target')

sns.scatterplot(x='s5', y='target', data=df, ax=axes[2])
axes[2].set_title('s5 vs target')
```

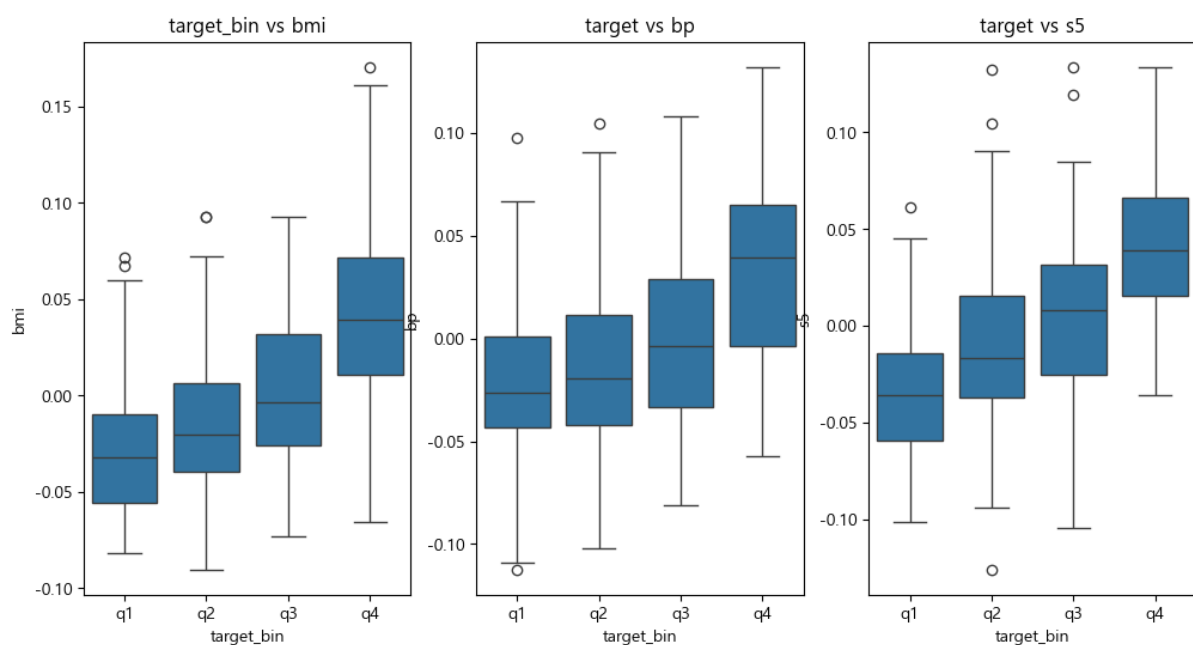
Scatterplot



```
# Scatterplot
sns.scatterplot(x='주요변수', y='목표변수', data=df, ax=axes
[0])
axes.set_title('')
```

■ x값에 따른 y값 추세 분포 확인 가능, x 증가에 따라 y도 증가하는 것을 볼 수 있음

boxplot 박스플롯



```
df['target_bin'] = pd.qcut(df['target'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
# pandas 내 pd.qcut 사용!
# q = 4로 labels를 붙이면 더 보기 쉬움
```

1. 분위수를 나눠서 범주형 변수 생성할 것!

- why? 수치가 너무 많음 -> 정리해줘야지

2. 파생변수 생성

- label 추가해주면 시각적으로 좋네

```
fig, axes = plt.subplots(1,3, figsize=(12,6))
sns.boxplot(x='target_bin', y='bmi', data=df, ax=axes[0])
axes[0].set_title("target_bin vs bmi")

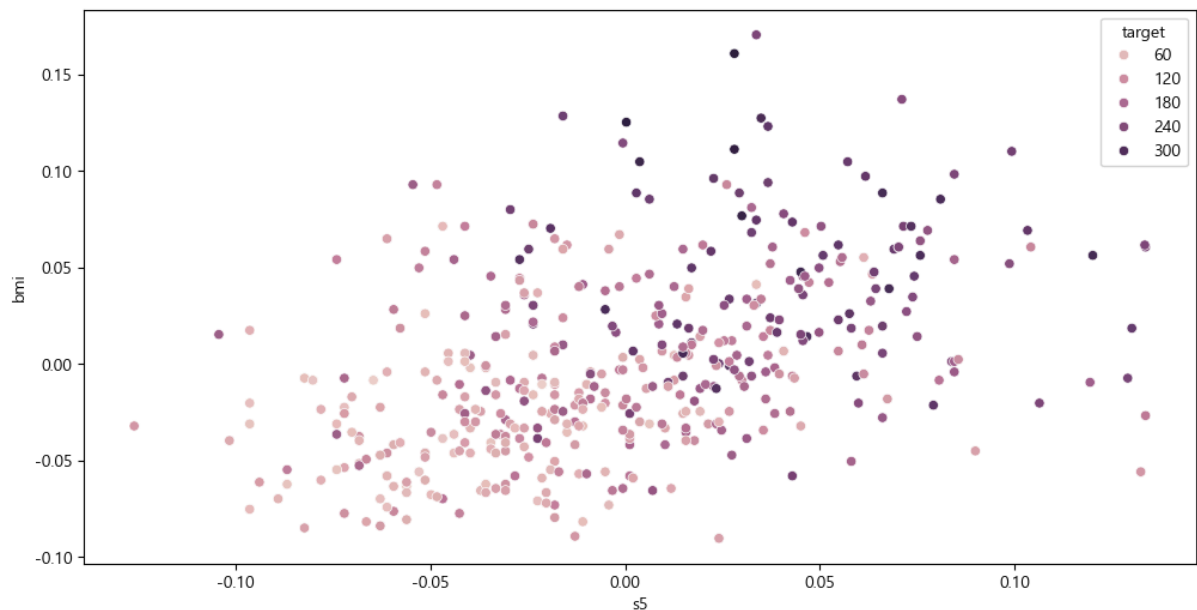
sns.boxplot(x='target_bin', y='bp', data=df, ax = axes[1])
axes[1].set_title("target vs bp")

sns.boxplot(x='target_bin', y='s5', data=df, ax = axes[2])
axes[2].set_title("target vs s5")
plt.show()
```

결과 해석 : 중앙값 증가, 이상치 확인 가능

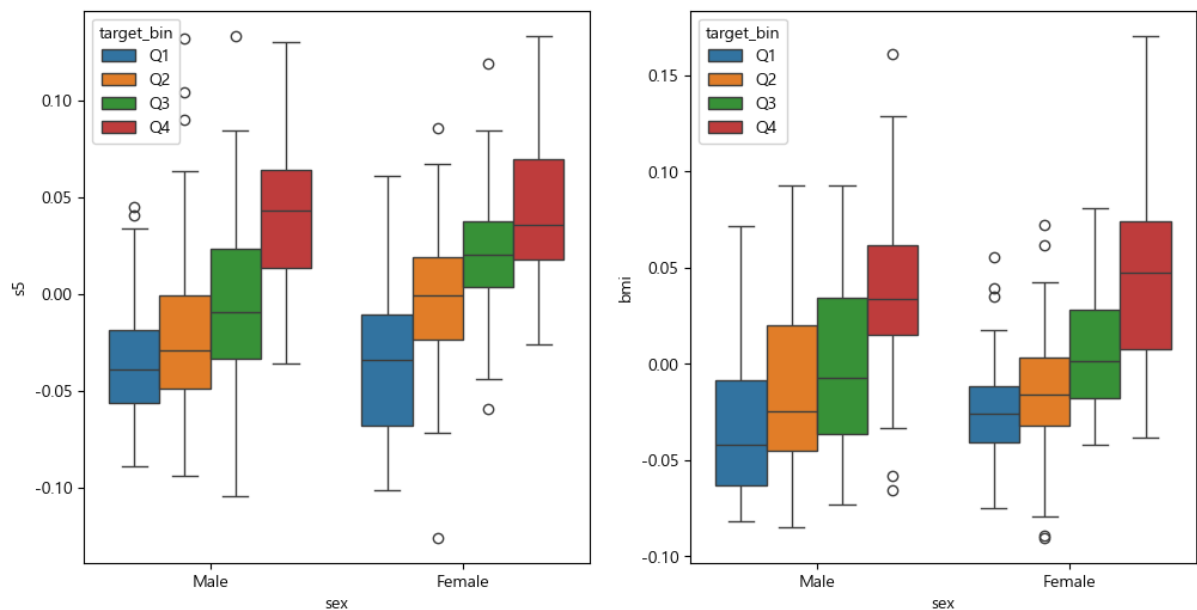
hue + scatterplot (다중 변수간의 상호작용 시각화)

```
fig, axse = plt.subplots(figsize=(12,6))
sns.scatterplot(data=df, x='s5', y='bmi', hue='target')
# x와 y를 두 개의 피쳐 조합으로 어떤 관계가 있는지 알기
```



결과 해석 : 두 변수 s5와 bmi가 높을수록 target의 분포가 300인 애들이 많이 찍힘

hue + boxplot



```
fig, axes = plt.subplots(1,2, figsize=(12,6))
sns.boxplot(data=df, x='sex', y='s5', ax=axes[0], hue='target_bin',)
sns.boxplot(data=df, x='sex', y='bmi', ax=axes[1], hue='target_bin')
```

```
axes[0].set_xticklabels(['Male', 'Female'])
axes[1].set_xticklabels(['Male', 'Female'])
```

결과 해석 : 당뇨병 환자 그룹별 성별에 따른 s5, bmi 박스플롯
BMI : 중앙값이 남성이 여성보다 높음

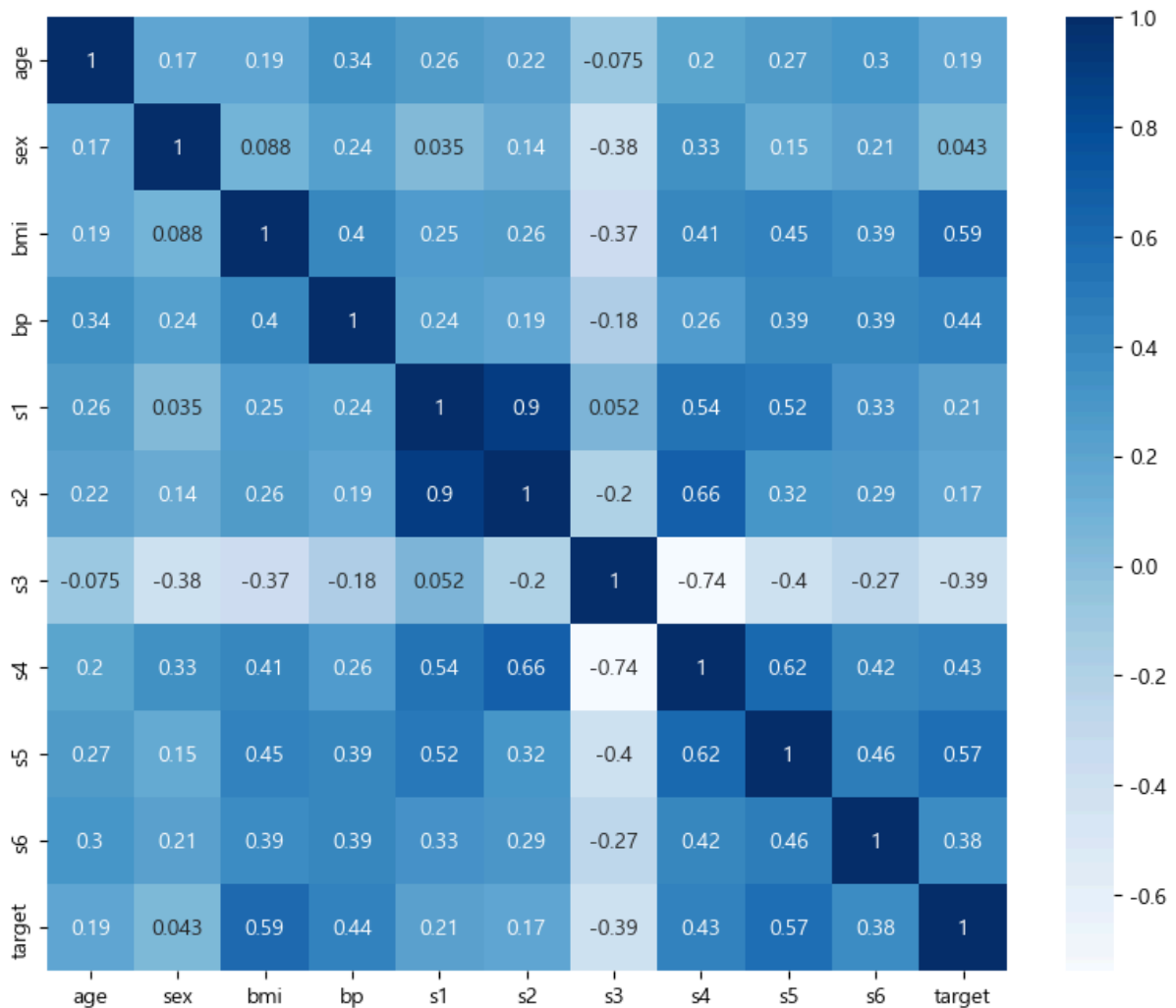
HeatMap

모든 변수 간 상관관계 행렬을 구하고 Heatmap을 구하고자할 때
모든 변수는 수치형 변수일 때 사용 가능!

	age	sex	bmi	bp	s1	s2	s3
age	1.000000	0.173737	0.185085	0.335428	0.260061	0.219243	-0.075181
sex	0.173737	1.000000	0.088161	0.241010	0.035277	0.142637	-0.379090
bmi	0.185085	0.088161	1.000000	0.395411	0.249777	0.261170	-0.366811
bp	0.335428	0.241010	0.395411	1.000000	0.242464	0.185548	-0.178762
s1	0.260061	0.035277	0.249777	0.242464	1.000000	0.896663	0.051519
s2	0.219243	0.142637	0.261170	0.185548	0.896663	1.000000	-0.196455
s3	-0.075181	-0.379090	-0.366811	-0.178762	0.051519	-0.196455	1.000000
s4	0.203841	0.332115	0.413807	0.257650	0.542207	0.659817	-0.738493
s5	0.270774	0.149916	0.446157	0.393480	0.515503	0.318357	-0.398577
s6	0.301731	0.208133	0.388680	0.390430	0.325717	0.290600	-0.273697
target	0.187889	0.043062	0.586450	0.441482	0.212022	0.174054	-0.394789

```
correlation_matrix = df.corr()
```

행렬표를 시각화하기!



- cbar : colorbar의 유무
- annot : 각 cell의 값 표기 유무 ****꼭해줄 것!****
- cmap = 'Blues' 히트맵의 색 설정

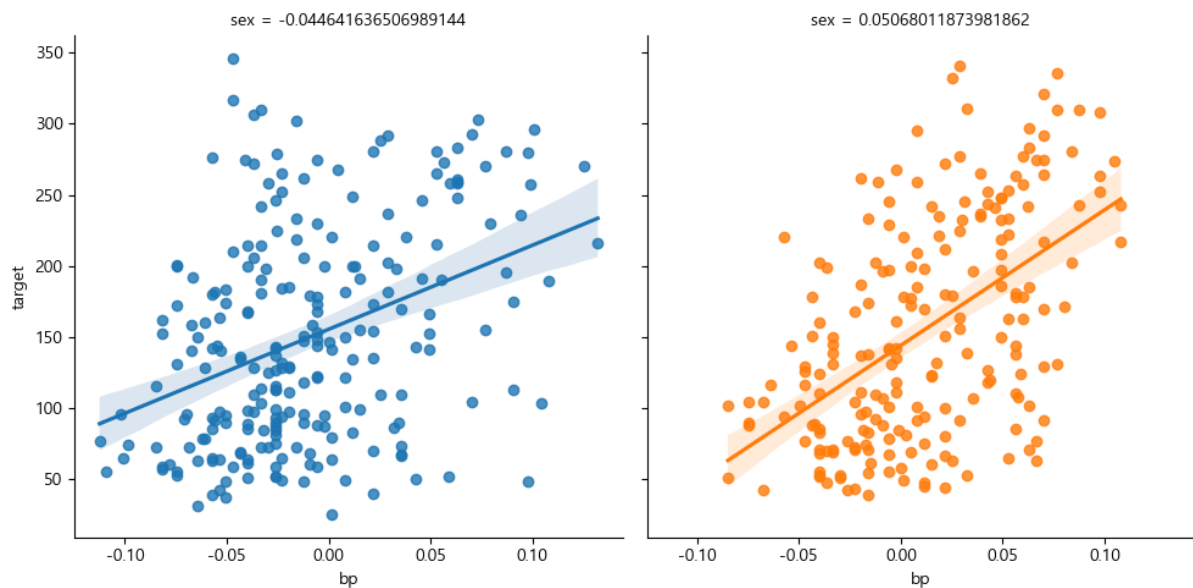
```
plt.figure(figsize=(10,8))
sns.heatmap(correlation_matrix,annot=True, cmap='Blues', cbar=True)
plt.show()
```

결과 분석 : 목표변수(target)과 같은 행 보기 → 상관계수가 제일 큰 칼럼 찾기

1. 0.59 bmi
2. 0.57 s5
3. 0.44 bp

Implot (회귀식 + 산점도)

- 산점도와 회귀선을 동시에 그려줌
- 신뢰구간을 음영으로 표시해줌
- hue 작동함
- subplots으로 넣기 불가능 ax가 안됨
- 범주형 변수의 경우 범주만큼 쪼개서 그리기 가능!!
 - 그럴 땐! hue + col 조합
 - hue로 범주를 지정하고
 - col로 동일한 범주명 지정





```
g = sns.lmplot(x = 'final_sec', y = 'split_frac', col='gender', data=data,
               markers=".", scatter_kws=dict(color='c'))
g.map(plt.axhline, y=0.1, color="k", ls=":");
```

Q : 회귀식을 보여주면 어떤 것을 설명할 수 있고 얘기할 수 있는가?

개수는 데이터를 많이 분포한다고 설명하기에 부적합하다고 생각함(얼마나 있는지 모름)

대신,

기울기가 급한 건 완주시간이 길어질 때 남성은 여성보다 positive split이 비율이 더 많다.

즉, 후반에 속도가 쳐지는 사람들이 남자가 더 많다?

KDE(sns.kedplot)와 hist(sns.hist) 변수 분포 확인 (확률 밀도 추정)

histogram (히스토그램)

```
# 1 sns.histplot
sns.histplot(data=df, x='target', bins=20, kde=True)
- KDE 구하기 가능
- y값을 구할 때 density로 보려면 stat='density'로 설정!

# 1.1 sns.histplot + stat = 'density'
sns.histplot(data['x'], kde=True, stat = 'density')
```

- 밀도가 나옴

```
# 2. plt 내장 모듈 사용
df.hist(bins=20, figsize=(12,6))
plt.show()

# 2.1 특정 칼럼만 구현
df[col].hist(bins=20)
plt.show()

# 3. plt.hist
plt.hist(data['x'], bins=20)

# 3.1 plt.hist + density=True
plt.hist(data['x'], bins=20, density=True)
- y값을 밀도로 보기 가능
```

sns. histogram 인자

- hist와 kde는 y가 필요 없음

```
### sns.histplot
- data : df 전체
- x = 칼럼 선택
- bins = 막대 개수
- kde (중요) = 밀도 추정 곡선
- color
- hue : 데이터 그룹을 나눠서 진행 가능
```

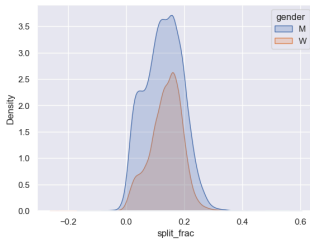
hue 인자

```
### hue
- 지정한 컬럼의 값(카테고리, 그룹, 또는 수치)에 따라 그래프의 색상을 다르게 표시합니다.
- 주 용도: 데이터 내에 그룹(예: 성별, 등급, 클래스 등)이 있을 때,
- 범주형 카테고리를 표현함
- 수치형일 경우 그라데이션 색상으로 구분
- multiple 속성을 추가하여 stack, dodge 등 막대 스타일을 조정 가
```

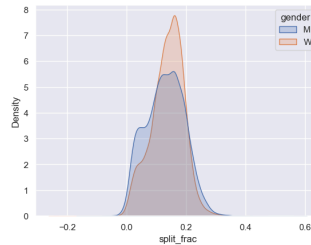
능함

- stack이 좀 구분됨
- hue를 사용하면 각 그룹간 상대적 분포 비교를 함, 면적 합이 1이라 면적이 넓은 만큼 분포 큼
- 개별 방식으로 하면 각 그룹의 분포가 면적이 1이기에 절대적인 모양, 상대적인 계산 안됨

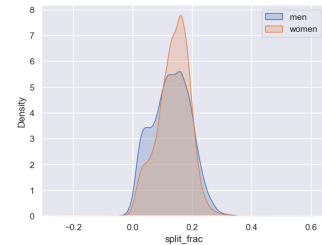
추가 중요 사실!!!



kdeplot으로 그린 값에
hue를 쓰면 전체 면적
합이 1로 나옴 (1)



제각각 그린 경우는 각
면적의 합이 1로 나옴
(2)



만약 hue를 사용한다면
common_norm=False
해야함 (3)

```
#1
sns.kdeplot(data=data, x='split_frac', hue = 'gender' , fill=True)

# 3
sns.kdeplot(data=data, x='split_frac', hue = 'gender' , fill=True, common_norm=False)

# 2
sns.kdeplot(data.split_frac[data.gender=='M'], label='men', fill=True)
sns.kdeplot(data.split_frac[data.gender=='W'], label='women', fill=True)
plt.xlabel('split_frac');
plt.legend()
```

결과 해석

1. hue 사용 시 (전체 합이1로 나오기에) 데이터 수가 많으면 밀도 높음 → 남성이 더 많이 있다
 2. hue 없이 하면, 각 면적이 1이기에 비교하기 좋음 → 남자는 여자보다 더 퍼져있음 분산이 큼
- 원하는 해석! 각각 1이라고 하면 데이터 수를 설명하기 보다 분산을 설명할 수 있음

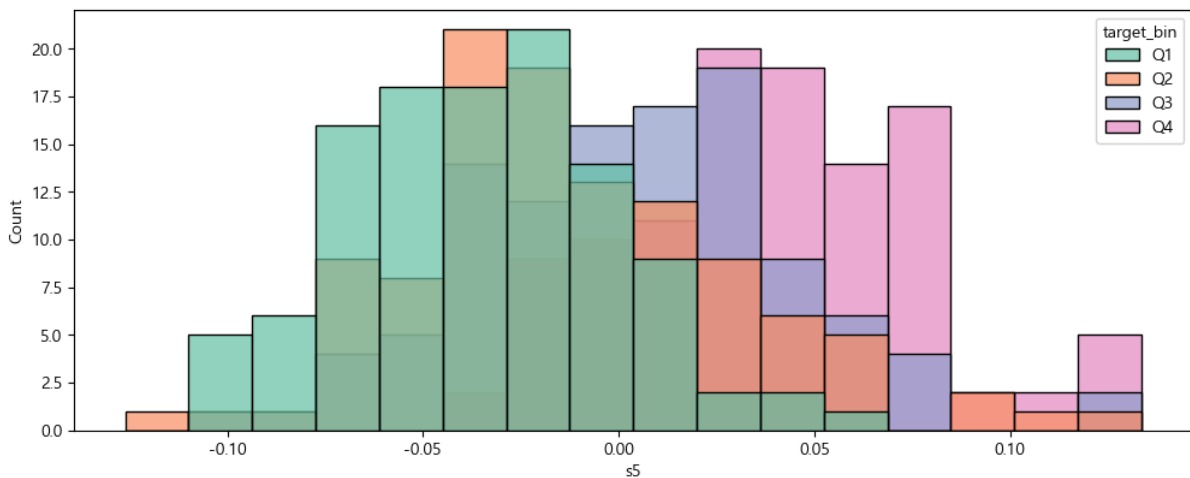
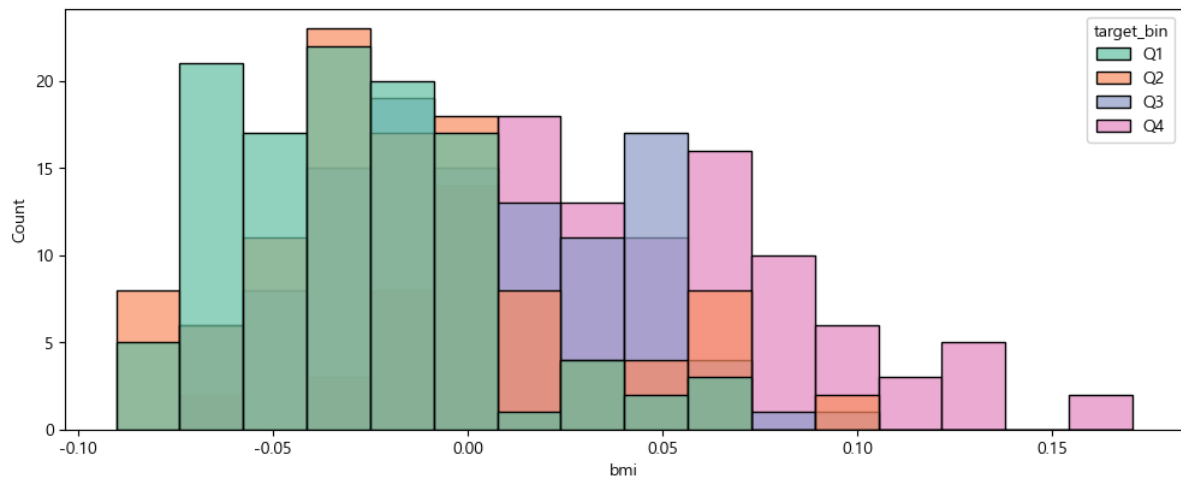
Q : target을 사분위수 기준 등으로 3 4개 구간으로 나누고, 각 구간별 주요 변수(bmi, s5)의 히스토그램 또는 KDE plot을 비교하여 시각화하시오.

kde. 타겟값의 구간별 **변수 분포** 비교를 하고자 할 때 사용.

- 분포를 찾을 때는 y값이 필요 없음 y값은 그냥 데이터 수임!

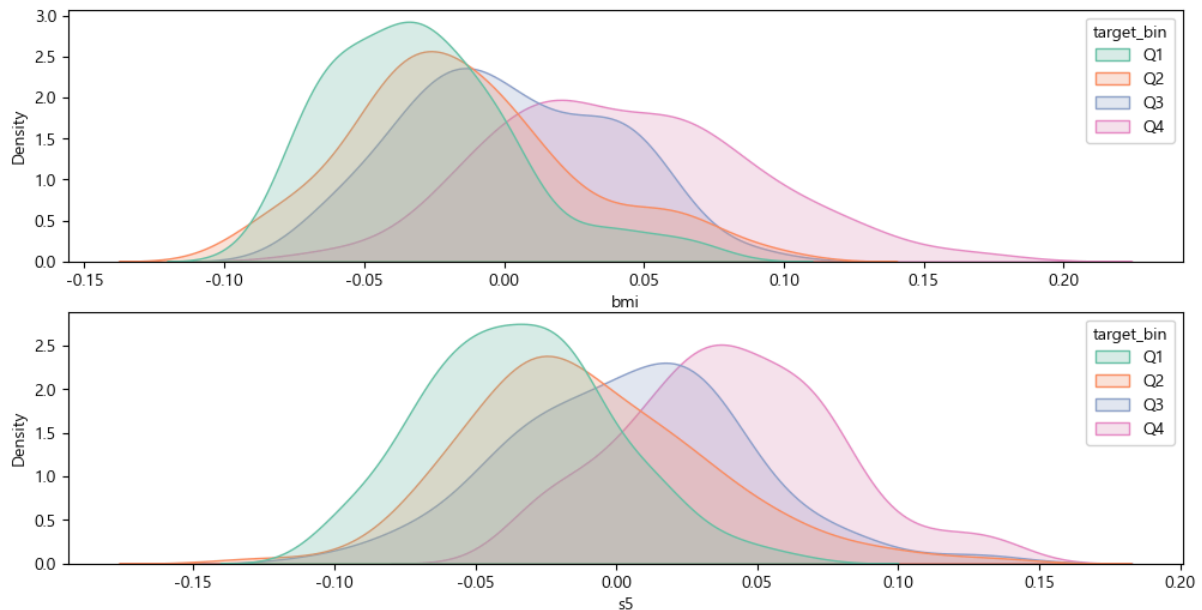
1) 타겟값 구간별 변수 분포를 구하라는 것 -> kde or hist를 구해라!

- hist와 kde는 sns을 사용해서 작성할 때, hue는 **구간별(목표값은)**, x는 **변수로 설정**



```
fig, axes = plt.subplots(2, 1, figsize=(12, 10))
sns.histplot(data=df, x='bmi', hue='target_bin', alpha=0.7,
palette='Set2', ax=axes[0])

sns.histplot(data=df, x='s5', hue='target_bin', alpha=0.7,
palette='Set2', ax=axes[1])
```



```
fig, axes = plt.subplots(2,1, figsize=(12,6))
sns.kdeplot(data=df, x='bmi', hue='target_bin', ax = axes[0],
fill=True, palette='Set2')
sns.kdeplot(data=df, x='s5', hue='target_bin', ax = axes
[1], fill=True, palette='Set2')
plt.show()
```

결과 해석

q1~q4 : 구간에 따라 다른 해석 가능
평균과 분산 중심으로 해석
오른쪽으로 갈 수록 평균값 크고 , 넓게 분포 꼬리가있을수록 분산이 크다.
→ Q4의 경우 s5와 bmi도 높은 사람들이 많음 q4는 심한 당뇨병 환자

BMI 측면

- Q4의 BMI 값은 다른 그룹들보다 확연히 높음 : 대부분의 값이 0~0.15 사이에 분포
- Q1과의 차이 : Q1은 주로 음수 값에 분포하는 반면, Q4는 대부분 양수 값에 분포

S5 측면 :

- Q4의 S5 값은 다른 그룹들과 가장 뚜렷하게 구분됨 : 피크가 약 0.05

부근에 위치

- Q1과 거의 겹치지 않음: S5 변수는 target 그룹들을 매우 명확하게 구분

plt만 써서 시각화 하기

커널 밀도 함수

- 커널 밀도

밀도 추정(density estimation)

- 전체가 아닌 부분의 데이터로 원래 변수의 확률 분포 특성을 추정하고자 하는 것
ex) 육교 밑을 통과하는 차량의 일일 교통량을 파악하자
변수 : 일일 교통량 데이터 : 500, 300, 450..
→ 이런 데이터가 쌓이면 일일 교통량이라는 변수가 어떤 분포를 가지는지 알 수 있다.
→ 또한 어떤 변수가 **가질 수 있는 값**과 **그 값을 가질 가능성**의 정도를 추적하는 것이 밀도 추정이다.

아하! 밀도추정을 통해 변수의 분포와 값 그리고 그 값을 가질 확률(확률밀도)을 구할 수 있구나!

- 확률 밀도 : Probability density
- 어떤 변수 x 의 밀도를 추정한다고 하면, x 의 확률밀도함수(pdf)를 추정하자는 것이다.

밀도 추정의 방법

- Parametric
 - 확률밀도함수에 대한 모델을 정해 놓고 데이터를 넣어 파라미터만 추정하는 것
 - PDF는 평균과 분산만을 구하면 구할 수 있음
- non- parametric
 - 확률밀도함수를 정하지 않고 진행
 - 데이터들을 히스토그램으로 구한 후 정규화하여 확률밀도함수로 사용
 - 히스토그램은! bins의 경계에서 불연속하고 여러 문제를 가지므로! 이를 해결하고자 **KDE**등장

커널 함수 정의방법

커널 함수 조건

1. 원점을 중심으로 좌우 대칭

2. 아래 세 조건 만족

$$\int k(x)dx = 1 \quad \int xk(x)dx = 0 \quad \int x^2k(x)dx > 0$$

커널 함수 $K(x)$ 에 대해서 KDE에서는 PDF(확률밀도함수)를 다음과 같이 정의한다.

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^N K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$
$$h = \frac{1}{M}$$

- 여기서 h 는 함수의 완만한 형태인지 뾰족한 형태인지 정하는 값으로
- 관측된 데이터마다 해당 데이터를 중심으로 하는 $K(x)$ 커널함수를 생성
- 이렇게 만들어진 커널 함수들을 모두 더한 후 전체 데이터 개수(n)로 나눈다.

히스토그램 함수 정의

- M : 빈의 개수
- n : 데이터의 개수
- h : 빈의 너비
 - 빈의 개수가 늘면 너비는 좁아짐
 - 너비가 커지면 빈은 줄어듦

$$\text{구간}[0, 1]$$
$$\hat{f}(x) = \frac{M}{n} \sum_{i=1}^n I(x_i \in B_j)$$
$$B_1 = [0, \frac{1}{M}], B_M = [\frac{M-1}{M}, 1]$$

▼ 히스토그램 & KDE 관련 문제

문제 1: 수식 해석

다음 두 밀도 추정식을 보고 물음에 답하시오.

$$\begin{aligned} \hat{f}(x) &= \frac{M}{n} \sum_{i=1}^n I(x_i \in B_j) \\ B_1 &= [0, \frac{1}{M}], B_M = [\frac{M-1}{M}, 1] \end{aligned} \quad \begin{aligned} \hat{f}_h(x) &= \frac{1}{n} \sum_{i=1}^N K_h(x - x_i) = \frac{1}{n} \sum_{i=1}^N \frac{1}{h} K(\frac{x - x_i}{h}) \\ h &= \frac{1}{M} \end{aligned}$$

- (A)와 (B)가 각각 어떤 밀도 추정 방법을 나타내는지 쓰시오.
 - 히스토그램 밀도 추정
 - 커널 밀도 추정
- 식 (A)에서 h 가 의미하는 바와 h 를 크게 할 때 발생하는 현상을 설명하시오.
 - 빈의 개수가 작아지면 h 빈의 너비는 커져 더 많은 데이터가 포함되고 히스토그램이 평평해진다.
- 식 (B)에서 커널 함수 $K(\cdot)$ 이 만족해야 하는 두 가지 수학적 조건을 서술하시오.
 - 적분값이 1이고
 - $xk(x)$ 의 적분값은 0이고
 - $x^2k(x)$ 는 0보다 크다

문제 2: 계산 실습

다음 데이터에 대해 히스토그램과 KDE를 수작업으로 계산하시오.

데이터: [1.2, 1.5, 1.7, 2.1, 2.3, 2.8, 3.0]

- 빈 너비(bin width) 0.5, 최소값 1.0으로 히스토그램 밀도를 계산하고 그래프를 그리시오.
bin = [1, 1.5], [1.5, 2.0] [2.0, 2.5] [2.5, 3.0] $h = 0.5$, $n = 7$ 개 { 빈의 개수 M 은 포함되는 개수임}
 M : 1, 2, 2, 2
- Bandwidth=0.4인 삼각 커널($K(u) = (1-|u|) \text{ for } |u| \leq 1$)을 사용해 $x=2.0$ 에서의 KDE 값을 계산하시오.

문제 3: 실제 데이터 분석

1. `bins=50` 을 `bins=5` 로 변경하면 KDE 곡선에 어떤 변화가 발생하는지

- KDE 곡선 자체는 변하지 않는다(히스토그램 bin 수와 무관).
- 히스토그램 막대만 더 두꺼워지고, 계단식이 더 거칠어진다.

2. `kde=True` 매개변수를 제거했을 때 시각화 결과

- KDE 곡선(부드러운 곡선)이 사라지고, 히스토그램만 남는다.

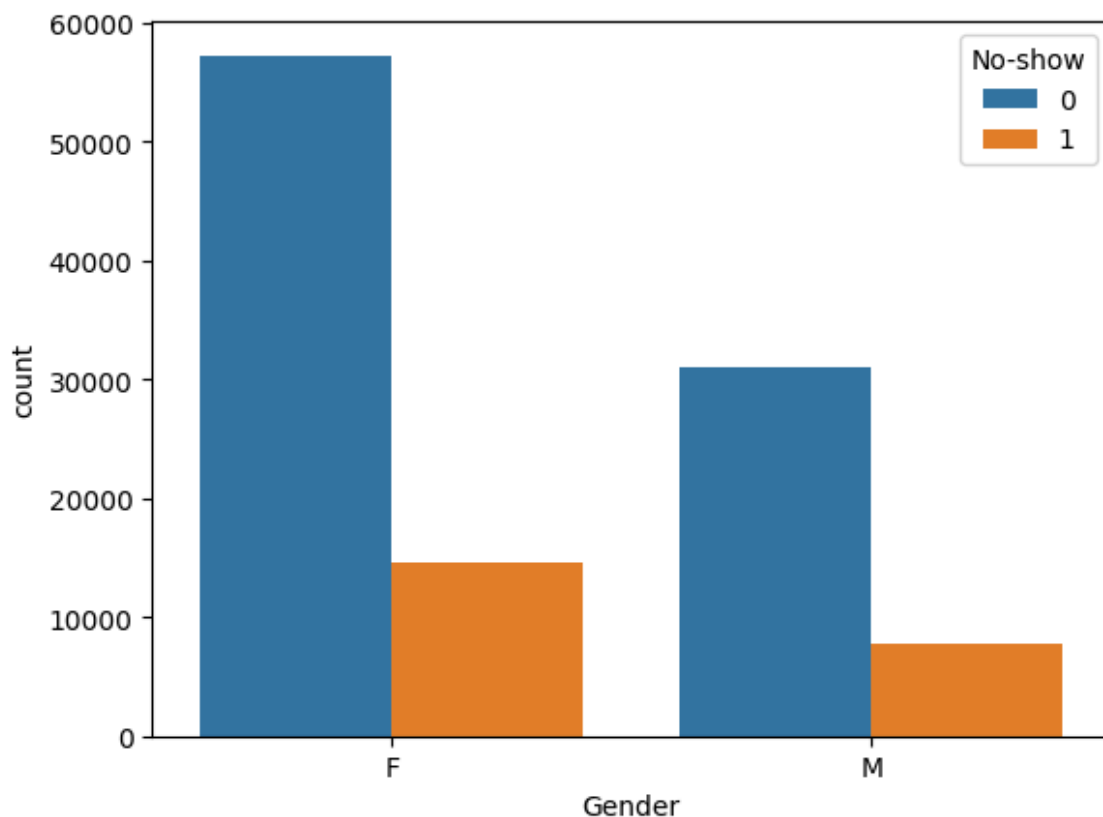
```
fig, axes = plt.subplots(2, 2, figsize=(8, 6))
axes[0,0].plot(x,np.sin(x), '-r')
```

코드

라이브러리

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

sns.countplot (막대그래프로 데이터간의 분포를 알고 싶을 때)



```
sns.countplot(x='Gender', hue='No-show', data=df)
```

plt.plot → label 사용가능

```
plt.plot(x, np.sin(x), linestyle='--', color='blue', label='sin(x)')
plt.plot(x, np.cos(x), linestyle='--', color=(1.0, 0.2, 0.3), label='cos(x)')
plt.xlim(-0.5, 10.5)
```

```
plt.ylim(-1.5,1.5)
plt.xlabel("x")
plt.ylabel('f(x)')
plt.legend()
```

참고(fig 저장 및 불러오기)

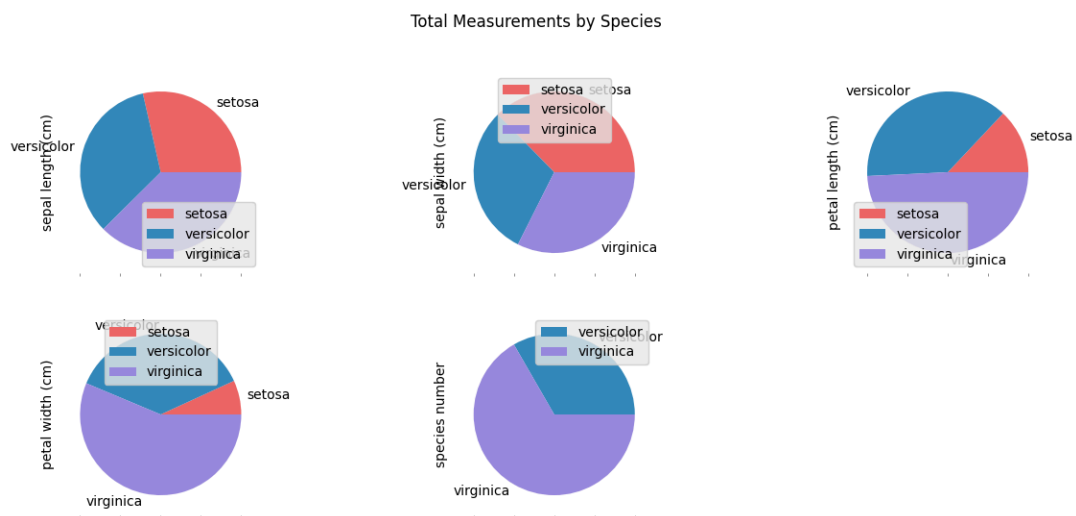
```
fig.savefig('my_figure.png')
from IPython.display import Image
Image('my_figure.png')
```

원형, 막대 등 차트는 plt.plot가 아닌 데이터셋.plot를 사용한다
기본은 데이터셋.plot(kind='bar','pie','scatter','ind')

원형 차트(pie)

```
sums_by_species['sepal length (cm)'].plot(kind='pie')
plt.suptitle('Total Measurements by Species')
```

- 데이터셋.plot(kind='pie')



- 데이터셋에 있는 칼럼이 여러개 이면
 - subplots=True + layout(행,열) 하면 여러 개 작성 가능

```
sums_by_species.plot(kind='pie', subplots=True, layout=(2,3), legend=True, figsize=(15,6))
```

```
plt.suptitle('Total Measurements by Species')
```

- plt.pie(data, labels(칼럼 내 unique))

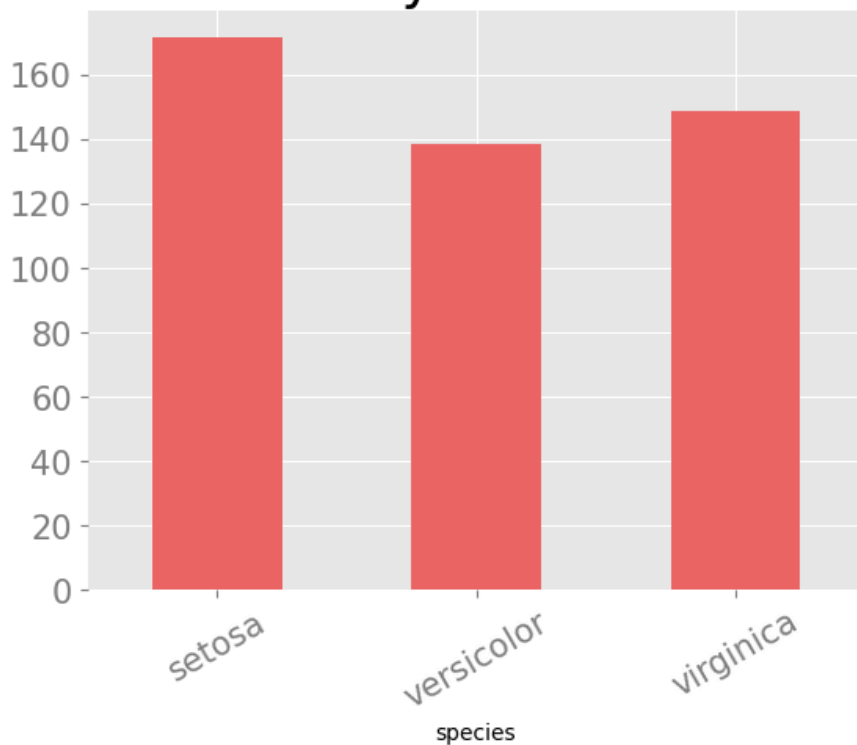
```
plt.pie(sums_by_species[var], labels=df['species'].unique(),
        colors = ['gold', 'yellowgreen', 'lightcoral'])
```

막대 그래프(bar)

```
df = df.groupby('species').sum()
col = 'sepal width'
df[col].plot(kind='bar')
```

- rot = 30 글자 각도

Iris classified by the width of sepal



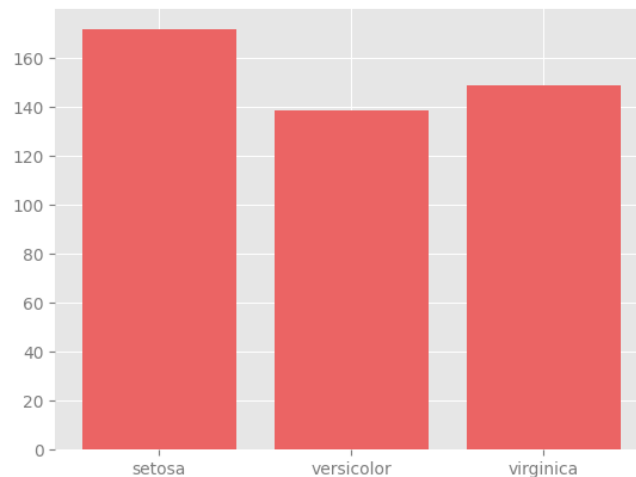
```
sums_by_species = df.groupby('species').sum()
```



```
var = 'sepal width (cm)'
sums_by_species[var].plot(kind='bar', fontsize=15, rot=30)

plt.title('Iris classified by the width of sepal', fontsize=25)
```

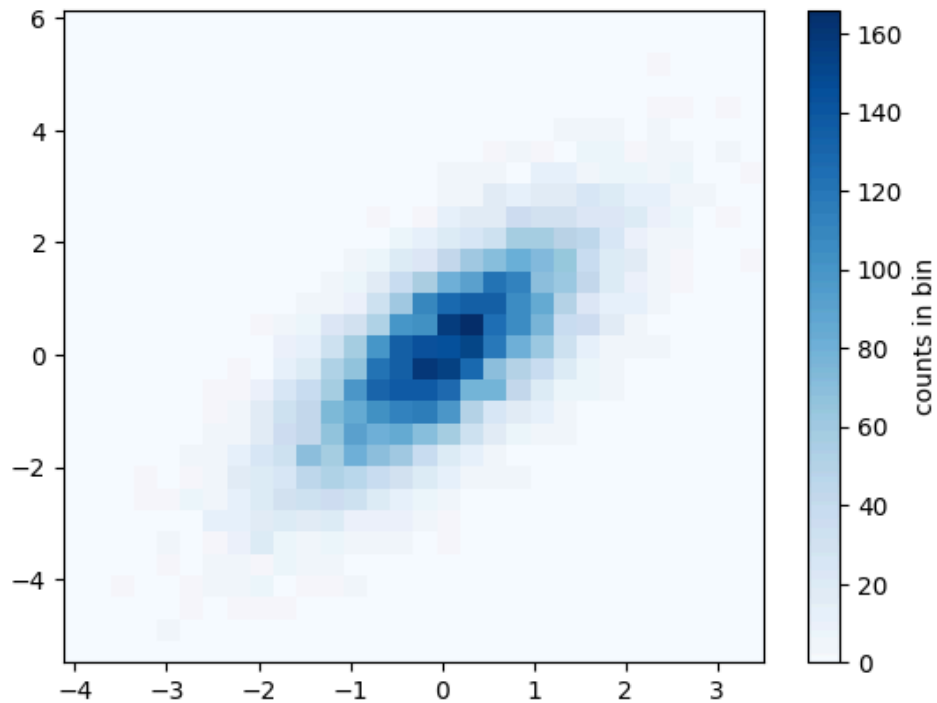
- `plt.bar` (x값(종류), y값은 수치)



```
plt.bar(sums_by_species.index, sums_by_species['sepal length (cm)'])

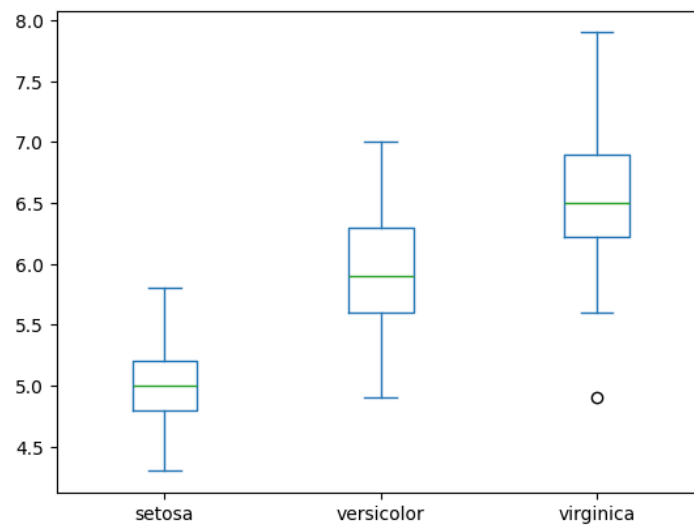
plt.bar(np.arange(3), sums_by_species[var], align='center')
plt.xticks(np.arange(3), df['species'].unique())
# 이렇게 x값을 붙여도 되는데 너무 귀찮지! 그냥 index함
```

plt.hist2d : 2차원 히스토그램



```
plt.hist2d(x, y, bins=30, cmap='Blues')
plt.colorbar()
```

상자그림(df.pivot(kind='ind', columns = '데이터칼럼', val = col)

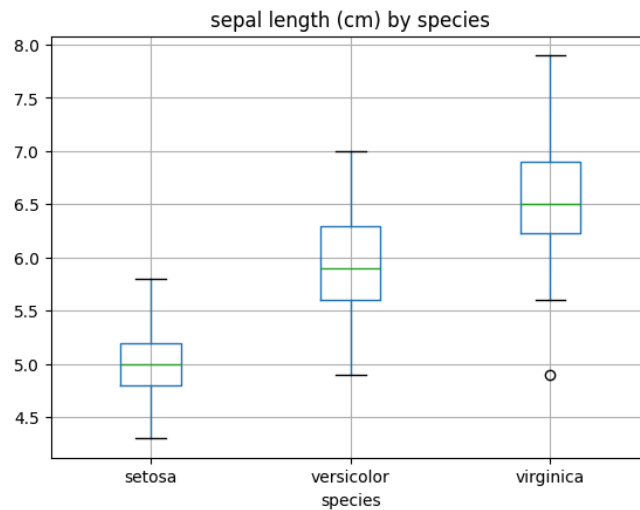


```
col = 'sepal length (cm)'
```

```
df['ind'] = pd.Series(df.index).apply(lambda i: i % 50)
df_pivot = df.pivot(index='ind', columns='species', val
```

```
ues=col)
df_pivot.plot(kind='box')
```

- df.boxplot(data , label=칼럼명, by='알고싶은 것')



```
df.boxplot(column=col, by='species')
plt.title(f'{col} by species')
plt.suptitle('')
plt.show()
```

행렬 산점도 from pandas.plotting import scatter_matrix)



```
from pandas.plotting import scatter_matrix  
df_iris = df[['sepal length (cm)': 'petal width (cm)']]  
scatter_matrix(df_iris)
```

Seaborn vs Matplotlib

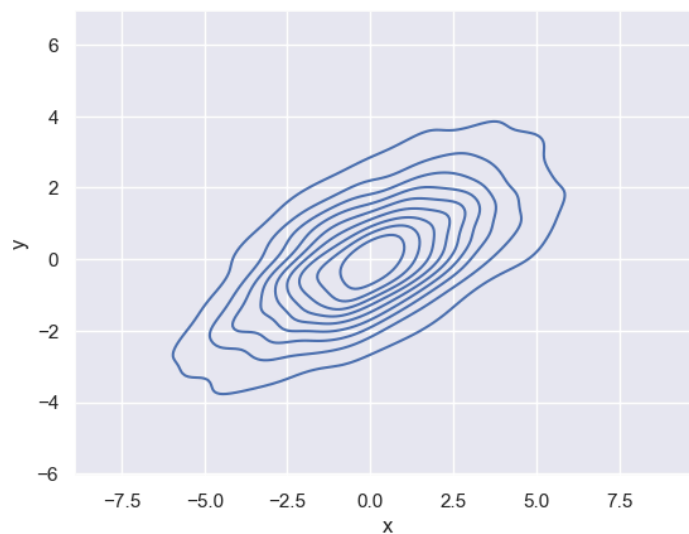
결합분포와 주변 분포

- 두 개 이상의 확률변수 (X,Y)가 동시에 특정 구간에 있을 확률분포
 - 이산형: $P(X=x, Y=y)$
 - 연속형: $f(x,y)$ (결합확률밀도함수)

예시 : 성별과 bmi같은 두 변수의 값이 동시에 특정 구간에 있을 확률

- 남성과 여성이 각각 Q1~Q4 그룹 내 bmi의 분포가 어떻게 다른가?
- 주변 분포 : 결합 분포에서 한 변수만 남기고 나머지 변수는 적분하여 단일변수 분포 구함

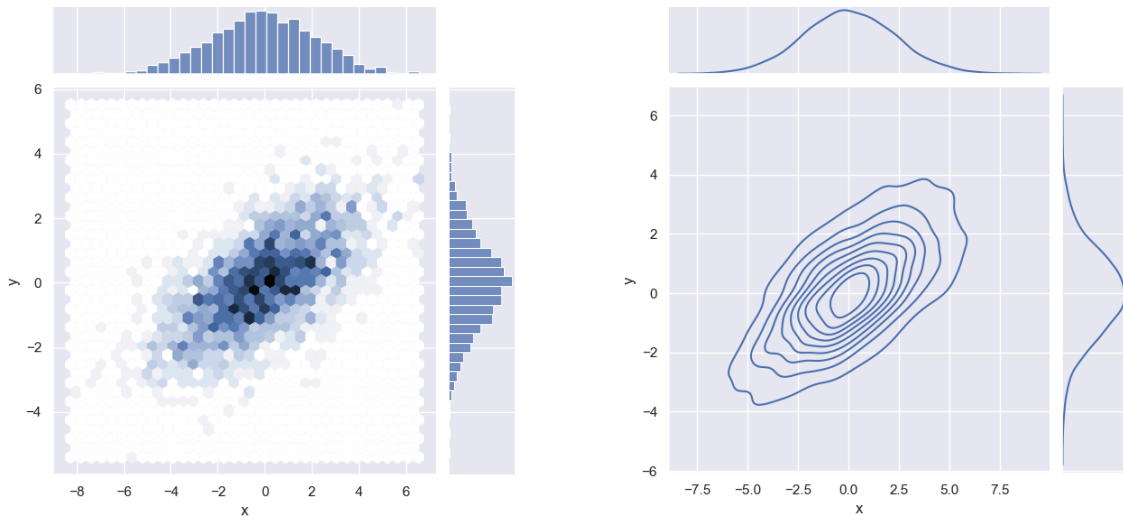
KDE를 2차원에서 그리면 나오는 분포 (결합분포)



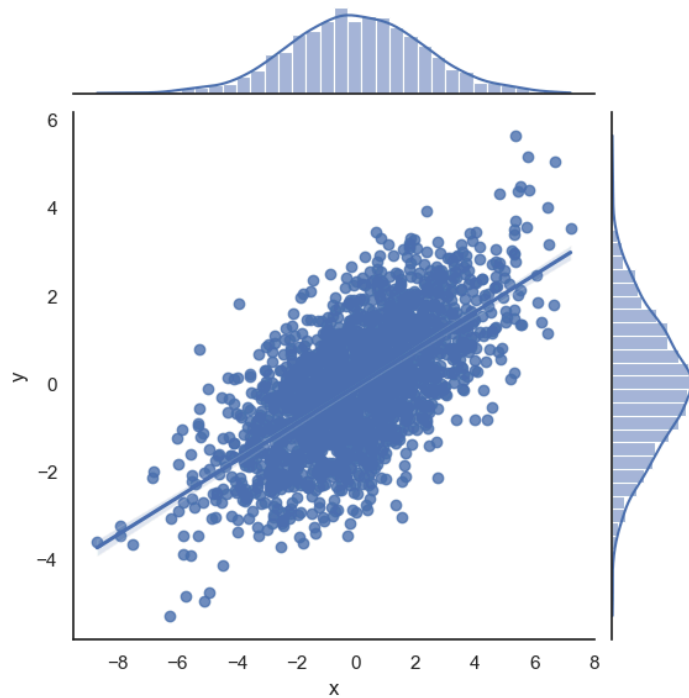
```
sns.kdeplot(x = data['x'], y = data['y'])
```

결합분포와 주변 분포 함께 그리기 (joinplot)

- 결합분포로 얼마나 분포하는지 보려면 kind='hex'로하기

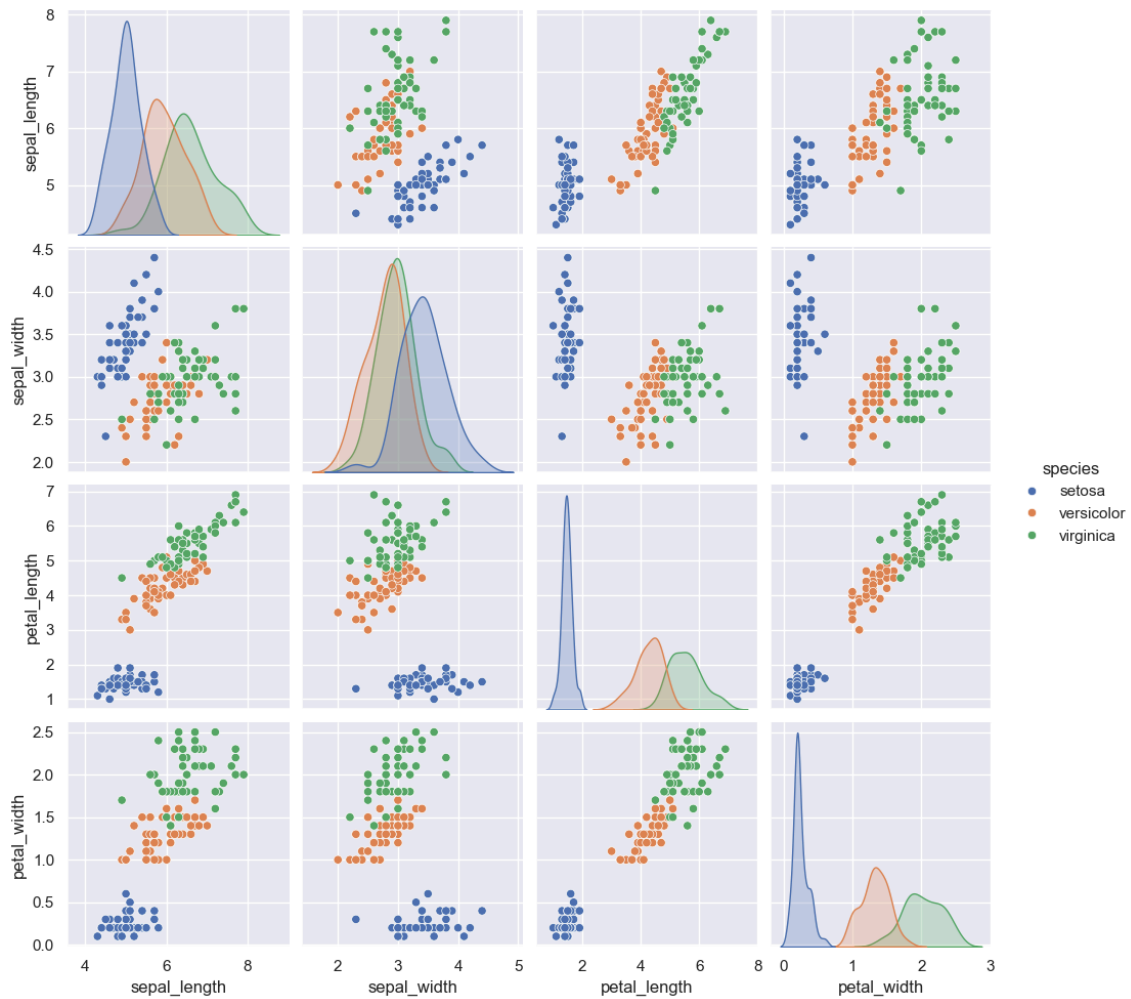


```
sns.jointplot(data, x = "x", y = "y", kind='hex')
sns.jointplot(data, x = "x", y = "y", kind='kde')
```



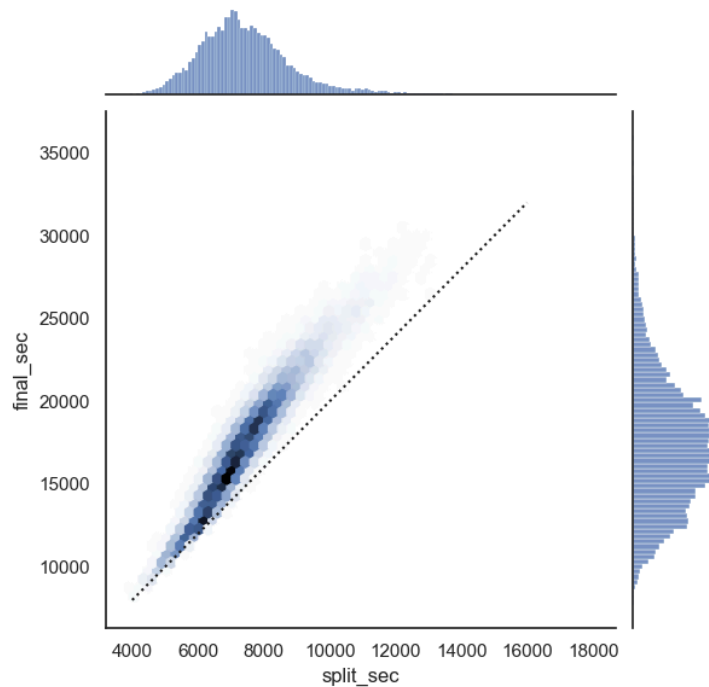
```
with sns.axes_style('white'):
    sns.jointplot(data, x = "x", y = "y", kind='reg')
    # reg regression 회귀선 줌
    # 상관관계 알려주는 것으로 점들이 붙어있으면 관계가 큼
```

행렬 산점도 2(pairplot)



```
sns.pairplot(iris, hue='species', height=2.5)
# hue로 종에 대해서 묶음
```

마라톤 데이터 분석



	age	gender	split	final	split_sec	final_sec
0	33	M	0 days 01:05:38	0 days 02:08:51	3938.0	7731.0
1	32	M	0 days 01:06:26	0 days 02:09:28	3986.0	7768.0
2	31	M	0 days 01:06:49	0 days 02:10:42	4009.0	7842.0
3	38	M	0 days 01:06:16	0 days 02:13:45	3976.0	8025.0
4	31	M	0 days 01:06:32	0 days 02:13:59	3992.0	8039.0

```
with sns.axes_style('white'):
    g = sns.jointplot(x = "split_sec", y = "final_sec", data = data, kind='hex')
    g.ax_joint.plot(np.linspace(4000, 16000),
                    np.linspace(8000, 32000), ':k')
    # y = 2x 그래프 그려서 확인하심
```

- 결합분포로 hex버전으로 그리고 분포가 몰려있는 곳은 색이 짙함
 - x : 중간 지점 기록 (split_sec)
 - y : 최종 기록 (final_sec)
 - 대각선을 그리는 방법은?
y=2x 그래프를 그려서 중간 지점의 기록에 따라 최종 기록을 비교할 수 있게 함
- 시간이기 때문에 숫자가 크다는 건 속도가 느리다

- 대부분이 대각선 위에 존재하므로 중간지점 시간 2배는 최종지점 기록임
 - $1/2 + 1/2 = 1$ 이니까!
- 2 * 중간 지점 기록 < 최종 지점 기록 ($y=2x+a$)
 - 포지티브 스플릿주자 : $y=2x$ 보다 위에 있는 경우, 대다수의 사람들임
 - 후반부가 길다 → 후반부에 느리게 달렸다.
- 2 * 중간 지점 기록 > 최종 지점 기록 ($y= 2x-a$)
 - 네거티브 스플릿주자
 - 중반까지는 천천히 달리고 최종때 빨리 달린 케이스임
- **시각화할때는 정규화를 통해 기준을 만들자! (파생변수 생성)**
 - 정규화 방법
 - 값의 크기 차이를 보정하기위해 (나중 - 초기)/초기화

$$\text{최종값} = \frac{y - 2x}{y}$$

$$\text{정규화} = \frac{\text{실제값} - \text{기대값}}{\text{기대값}}$$

팁

데이터프레임에서 원하는 칼럼만 추출하는 방법

- 괄호를 두 개. 쓰자

```
df[['sepal length (cm)', 'sepal width (cm)']]
```

1. 칼럼 분리하기 (일일이 할순 있지만, 변수 선언해서 한 번에 가능)

- a. numerical_col = df.select_dtypes(include=['int64', 'float64']).columns.tolist()

2. data 타입 변경

- a. 수치형 변수 → 범주형 변수

- i. df['No-show'] = df['No-show'].map({'Yes':1, 'No':0})

- b. 날짜형 변수

- i. df['AppointmentDay'] = pd.to_datetime(df['AppointmentDay'])

- ii. df['waiting_day'] = df['AppointmentDay'].dt.dayofyear - df['ScheduledDay'].dt.dayofyear

3. 이상치 탐지 후 제거

- a. boxplot 시각화

- b. q1, q3 정의

Q1 = df['컬럼명'].quantile(0.25)

Q3 = df['컬럼명'].quantile(0.75)

IQR = Q3 - Q1

- c. 경계 산정

lower_limit = Q1 - 1.5 * IQR

upper_limit = Q3 + 1.5 * IQR

- d. 이상치 제거 후 박스 플롯

- a. df = df[(df['컬럼명'] ≥ lower_limit) & (df['컬럼명'] ≤ upper_limit)]

- b. sns.boxplot(data=df, x=col)

특정 칼럼명 가져오기

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species number	ind
species						
setosa	250.3	171.4	73.1	12.3	0	1225
versicolor	296.8	138.5	213.0	66.3	50	1225
virginica	329.4	148.7	277.6	101.3	100	1225

```
sums_by_species = df.groupby('species').sum()
```

```
sums_by_species.vali
```

```
sums_by_species.col
```

```
sums_by_species.ind
```